

Practical : 5	Implementation of 0/1 Knapsack Problem using Dynamic Programming.
Date : 12/08/2026	

Objective :- To implement the 0/1 Knapsack Problem in C++ using Dynamic Programming to determine the maximum value achievable within a given weight capacity, and to analyse its time and space complexity.

Program :-

```
#include <iostream>
#include <vector>
using namespace std;

// ----- 0/1 Knapsack (Dynamic Programming, O(nW)) -----
int knapsack01(int W, vector<int>& wt, vector<int>& val, int n) {
    vector<vector<int>>> dp(n + 1, vector<int>(W + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w]; // exclude item i
            if (wt[i - 1] <= w)
                dp[i][w] = max(dp[i][w], dp[i - 1][w - wt[i - 1]] + val[i - 1]);
        }
    }
    return dp[n][W];
}

int main() {
    vector<int> val = {60, 100, 120, 80};
    vector<int> wt = {10, 20, 30, 15};
    int W = 50;
    int n = val.size();

    int best = knapsack01(W, wt, val, n);
    cout << "Maximum value in knapsack of capacity " << W << ": " << best << endl;
    return 0;
}
```

Output :-

Maximum value in knapsack of capacity 50: 240

Analysis:-

Time & Space Complexity (n = number of items, W = knapsack capacity):

Method	Time Complexity	Space Complexity
0/1 Knapsack - Dynamic Programming	O(nW)	O(nW)
0/1 Knapsack - Space-Optimised DP	O(nW)	O(W)

The 0/1 Knapsack problem asks for the maximum total value of items that fit within a weight capacity W , where each item may be taken at most once. The DP table $dp[i][w]$ stores the best value achievable using the first i items with capacity w , built by choosing the better of excluding item i ($dp[i-1][w]$) or including it ($dp[i-1][w-wt[i-1]] + val[i-1]$) whenever it fits, so filling the full $n \times W$ table costs $O(nW)$ time and $O(nW)$ space. Because each row of the table only ever depends on the row above it, the space can be reduced to $O(W)$ by iterating the weight dimension from high to low and reusing a single 1-D array, without changing the $O(nW)$ time bound. For the sample instance with items of value $\{60, 100, 120, 80\}$ and weight $\{10, 20, 30, 15\}$ in a capacity-50 knapsack, the DP correctly finds that items 2 and 3 (weights 20 and 30, values 100 and 120) exactly fill the capacity for a maximum value of 240, better than any other combination.

Conclusion :- The 0/1 Knapsack Problem was successfully implemented in C++ using a bottom-up Dynamic Programming table over items and capacity. The program correctly computed the maximum achievable value of 240 for the sample instance, confirming the expected $O(nW)$ time and $O(nW)$ space (reducible to $O(W)$ with a rolling array) behaviour of the DP approach. This pseudo-polynomial complexity is efficient for moderate capacities but grows with W , which is why 0/1 Knapsack is classified as weakly NP-hard rather than solvable in strictly polynomial time.